

JWT Authentication in Django: How Login Actually Works (After Months of Confusion)

6 min read · Dec 18, 2025



Oviyan S

Follow



Listen



Share

So there I was, building my first real Django app. Everything was going great until I hit the authentication part.

“Just add login functionality,” my project manager said casually, like it was adding a button.

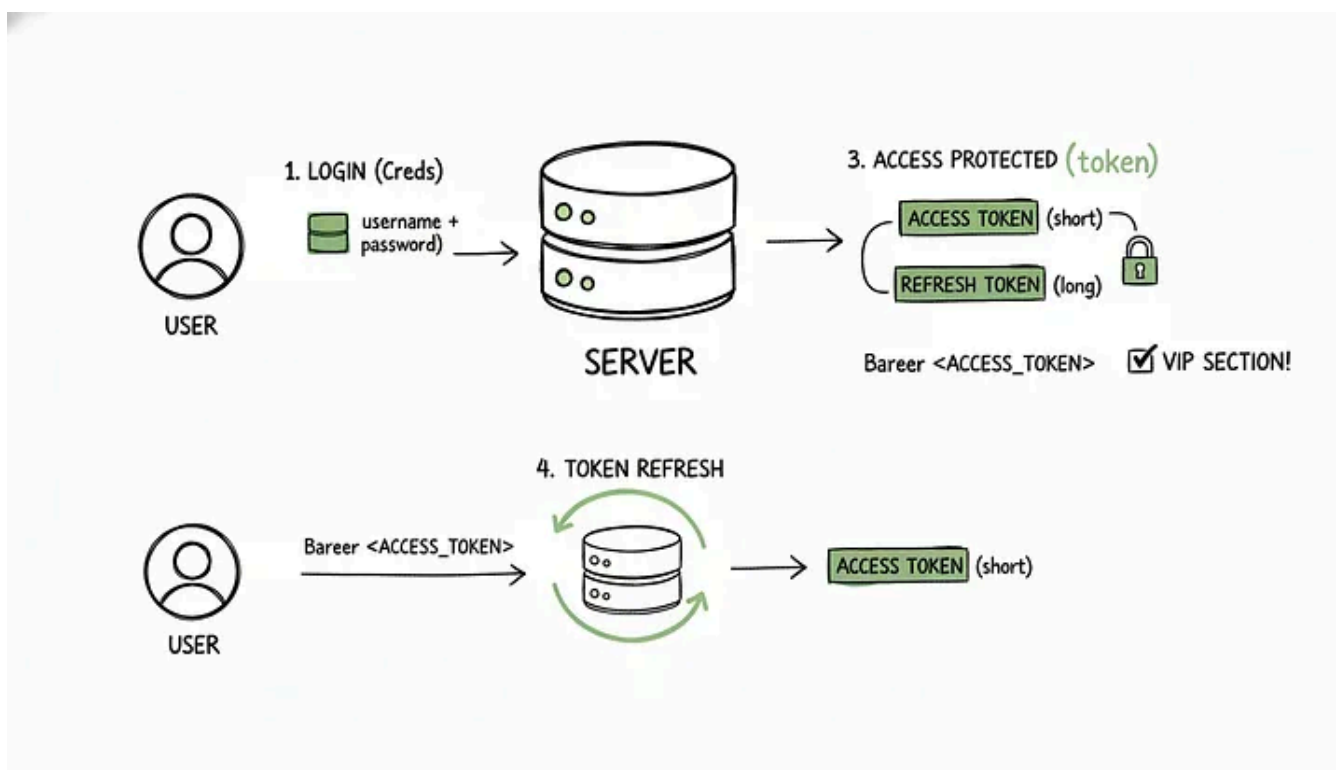
I Googled “Django authentication.” Found a tutorial. Copied the code. It worked... sort of. Then I tried adding a React frontend. Everything broke. Users couldn’t stay logged in. Tokens disappeared. Sessions vanished into thin air.

I spent three days debugging before I realised: **I had no idea how authentication actually worked.**

I was copying code like a monkey pressing buttons, hoping something would stick.

That’s when my senior dev friend sat me down and said, “Let me show you how JWT authentication actually works. Not the copy-paste version. The real version where you understand what’s happening.”

This is that conversation, turned into a tutorial.



JWT Authentication in Django: How Login Actually Works

Part 1: Backend Only (We'll Add React Later)

Before we touch any frontend code, we're building the backend properly. Think of it like building a house — you don't start decorating the living room before the foundation is poured.

In this part, we're building:

- User registration (sign up)
- User login (get tokens)
- Protected routes (members-only areas)
- Token refresh (stay logged in without re-entering password)

All tested with Postman, no frontend needed yet.

What Is Authentication Anyway?

friend started simple: "Authentication is just answering one question: Who are you?"

Think of it like a nightclub:

1. **Registration:** You sign up for a membership card
2. **Login:** You show your ID at the door

3. **Access:** Bouncer checks your ID and lets you in

4. **Protected Areas:** VIP section requires that same ID

No ID = no entry. Fake ID = kicked out. Simple.

In web apps:

- Username + password = your ID
- Server checks if they match = bouncer verification
- If valid → you get access
- If invalid → door stays closed

What Makes JWT Different?

“Okay,” My friend said, “here’s where it gets interesting. Traditional authentication is like getting a hand stamp at a club. You show your ID once, get stamped, and the bouncer recognises the stamp all night.”

“But that stamp only works at THAT club, right?” I asked.

“Exactly! And the club needs to remember everyone’s stamp. That’s a session — the server stores who’s logged in.”

“JWT is different,” she continued. “Instead of a stamp, you get a special wristband with encrypted information. The wristband itself proves who you are. The bouncer doesn’t need to remember you — they just check if the wristband is valid.”

Key difference:

- **Sessions:** Server remembers you (stores data in database)
- **JWT:** Server doesn’t remember you (you carry proof with you)

This makes JWT perfect for modern apps with separate frontends and backends.

The Two Tokens: Access and Refresh

“JWT gives you two wristbands,” Myfriend explained.

Access Token (Short-lived, like 5 minutes)

- This is your main wristband
- You show it to access protected areas
- Expires quickly for security
- Think: temporary entry pass

Refresh Token (Long-lived, like a day)

- This is your backup wristband
- When your access token expires, use this to get a new one
- Like having a VIP card that lets you get new entry passes
- Expires eventually, forcing you to log in again

“Why not just one long-lasting token?” I asked.

“Security,” the friend said. “If someone steals your access token, it’s only valid for 5 minutes. If they steal your refresh token... well, that’s worse, but at least your access token changes frequently.”

Makes sense. Multiple layers of security.

Let’s Build This Thing

Step 1: Setting Up the Kitchen

First, we need our workspace to be clean and organised.

Create your project structure:

```
chatApplication/  
├─ backend/  
└─ frontend/ (we'll tackle this in Part 2)
```

Navigate to backend, create a virtual environment (your isolated workspace where nothing can mess with other projects):

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

bash

```
cd backend
py -m venv .venv
.venv\Scripts\activate # Windows
# or: source .venv/bin/activate # Mac/Linux
```

Install the tools we need:

```
pip install django djangorestframework djangorestframework-simplejwt
```

Think of these as:

- **Django:** The restaurant building
- **Django REST Framework:** The kitchen equipment for serving data
- **Simple JWT:** The wristband printing machine

Step 2: Create the Restaurant

```
django-admin startproject chatApplication .
python manage.py startapp core
```

Now tell Django about our new tools in `settings.py` :

```

INSTALLED_APPS = [
    # Django's built-in stuff
    'django.contrib.admin',
    'django.contrib.auth',
    # ... other defaults

    # Our new tools
    'rest_framework',
    'rest_framework_simplejwt',
    'rest_framework_simplejwt.token_blacklist',

    # Our app
    'core',
]

```

Step 3: Configure the Wristband Machine

Still in `settings.py`, add this:

```

from datetime import timedelta
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
}
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=5),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=1),
    'AUTH_HEADER_TYPES': ('Bearer',),
}

```

Translation:

- Access tokens expire in 5 minutes
- Refresh tokens expire in 1 day
- We'll send tokens in the format: `Bearer <token>`

Step 4: Set Up the Database

```
python manage.py makemigrations
python manage.py migrate
```

This creates the tables where user data lives. Like building the filing cabinet before storing papers.

Building Registration (The Membership Sign-Up)

The Registration Form (Serializer)

Create `core/serializers.py`:

```
from rest_framework import serializers
from django.contrib.auth.models import User

class RegisterSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True)

    class Meta:
        model = User
        fields = ['username', 'email', 'password']

    def create(self, validated_data):
        user = User.objects.create_user(
            username=validated_data['username'],
            email=validated_data.get('email'),
            password=validated_data['password']
        )
        return user
```

Friend explained: “This is like the membership application form. It defines what information we need (username, email, password) and how to process it.”

The `write_only=True` on password means: "Accept the password, but never send it back in responses." Security 101.

The Registration Desk (View)

Create `core/views.py`:

```
from rest_framework import generics, status
from rest_framework.response import Response
from .serializers import RegisterSerializer
```

```

class RegisterView(generics.CreateAPIView):
    serializer_class = RegisterSerializer

    def create(self, request, *args, **kwargs):
        serializer = self.get_serializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        serializer.save()
        return Response(
            {"message": "User registered successfully"},
            status=status.HTTP_201_CREATED
        )

```

“This is the person at the registration desk,” Friend said. “They take your application, validate it, and create your account.”

The Sign (URL)

Create `core/urls.py`:

```

from django.urls import path
from .views import RegisterView
urlpatterns = [
    path('register/', RegisterView.as_view(), name='register'),
]

```

And connect it in main `chatApplication/urls.py`:

```

from django.contrib import admin
from django.urls import path, include
from rest_framework_simplejwt.views import (
    TokenObtainPairView,
    TokenRefreshView,
)
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/auth/', include('core.urls')),

    # JWT login endpoints (provided by Simple JWT)
    path('api/token/', TokenObtainPairView.as_view()),
    path('api/token/refresh/', TokenRefreshView.as_view()),
]

```

Testing Registration

Fire up the server:

```
python manage.py runserver
```

Open Postman, make a POST request to:

```
http://localhost:8000/api/auth/register/
```

Body (JSON):

```
{  
  "username": "testuser",  
  "email": "test@example.com",  
  "password": "securepassword123"  
}
```

Hit send. If you see “User registered successfully,” congratulations! Your registration desk is open for business.

Building Login (Getting Your Wristbands)

Here’s the beautiful part: Simple JWT already built the login for us. We just need to use it.

POST request to:

```
http://localhost:8000/api/token/
```

Body:

```
{
  "username": "testuser",
  "password": "securepassword123"
}
```

Response:

```
{
  "access": "eyJ0eXAiOiJKV1QiLCJhbGciOi...",
  "refresh": "eyJ0eXAiOiJKV1QiLCJhbGciOi..."
}
```

Those long strings? Those are your wristbands. The access token is your main pass. The refresh token is your backup.

Protecting a Route (The VIP Section)

Add this to `core/views.py`:

```
from rest_framework.views import APIView
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
class ProtectedView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request):
        return Response({
            "message": f"Hello {request.user.username}, you're in the VIP section"
        })
```

Add the URL in `core/urls.py`:

```
path('protected/', ProtectedView.as_view()),
```

Testing the VIP Section

Try accessing it without a token:

```
GET http://localhost:8000/api/auth/protected/
```

Response: “Authentication credentials were not provided.”

The bouncer kicked you out.

Now try WITH your access token:

```
GET http://localhost:8000/api/auth/protected/  
Authorization: Bearer <your_access_token_here>
```

Response: “Hello testuser, you’re in the VIP section!”

The bouncer checked your wristband and let you in.

What We Just Built

Let’s recap what you now have:

- User registration working
- User login returning JWT tokens
- Protected routes that require authentication
- Token validation happening automatically
- Backend authentication fully functional

Friend smiled. “See? When you understand the flow, it’s not magic. It’s just checking wristbands.”

What’s Coming in Part 2

In the next part, we’ll build the React frontend that talks to this backend:

- Login and registration forms

- Storing tokens securely
- Sending tokens with every request
- Automatic token refresh when the access token expires
- Logout functionality

But for now, you have a working backend. Test it. Break it. Understand it.

Because the best way to learn isn't copying code — it's understanding why each piece exists.

I've put together a list of 150+ free, open-source AI tools [grab it here.](#)

Jwt Token

Python

Django Rest Framework

React

Tutorial



Follow

Written by Oviyan S

101 followers · 22 following

Passionate tech enthusiast Solving real-world problems with Django and React Js

Responses (1)



Write a response

What are your thoughts?

Roberto Henriques
Jan 25



Hi Ovlan,

I liked very much your article about JWT and the way you explained it. I just want to know if there is an way to have this kind of authentication when just using only Django and its templates without React.

Could you help me with that?



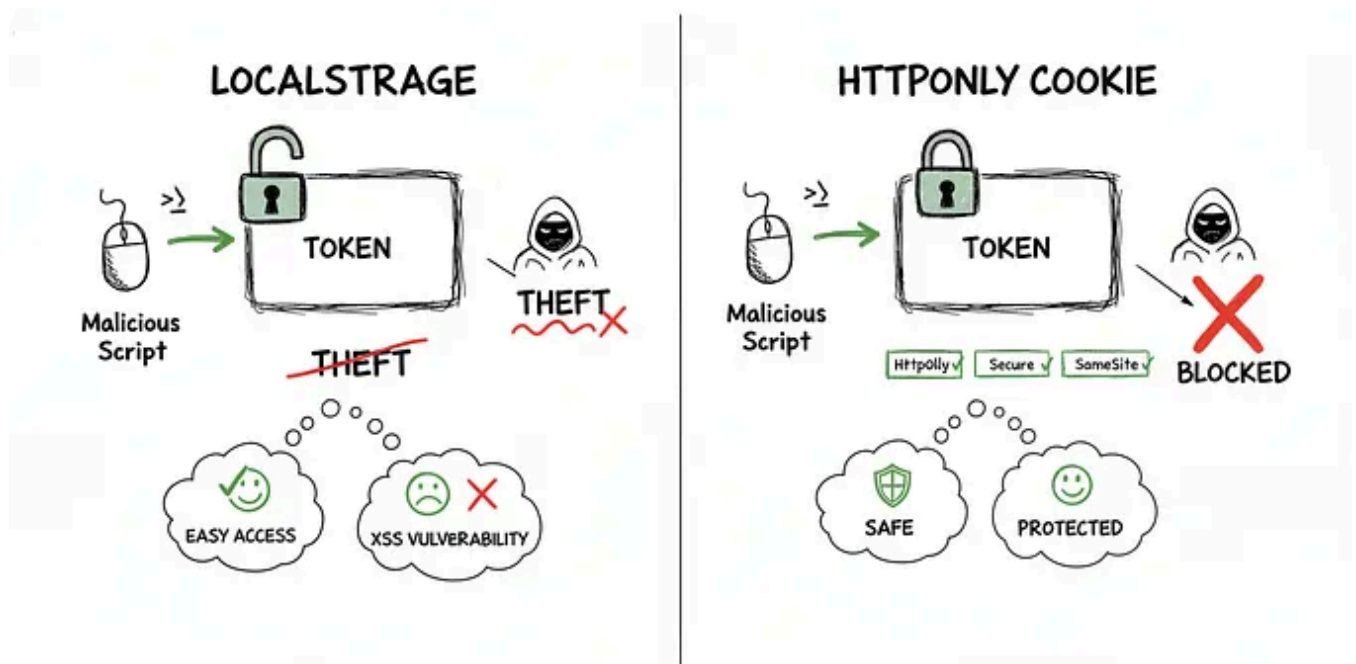
2



1 reply

[Reply](#)

More from Oviyan S



Oviyan S · Feb 12

How to Securely Store JWT Tokens in Cookies (Avoid XSS Risks)

So there I was, feeling pretty proud of myself. I'd just finished building a full-stack authentication system with Django and React. JWT...



8





How to Deploy Your Django and React app On Nginx

Open in app ↗

Sign up

Sign in

Medium

Search



Read more on medium.com /@oviyan007

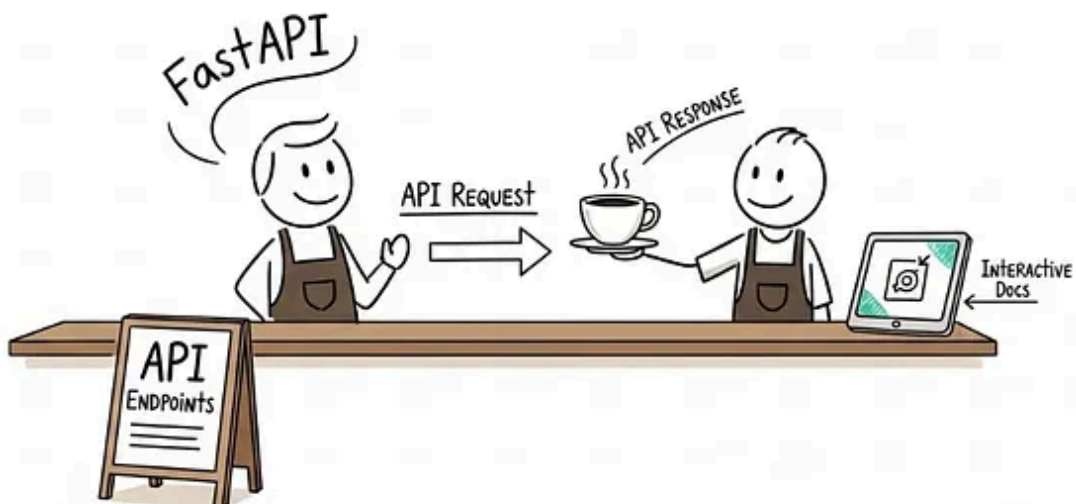


Oviyan S · Jun 9

From Localhost to Production: Deploying Django + React with HTTPS and Nginx

The “Oh Crap, Now What?” Moment

2



In CodeX by Oviyan S · Oct 9, 2025

FastAPI Tutorial Part 1: Your Coffee Shop Opens for Business

Why Your First API Feels Like Learning to Drive (And How We'll Fix That)



PY In Python in Plain English by Oviyan S · Sep 25, 2025

How I Automated YouTube Search and Play Using Playwright (Python)

You know that moment when you open YouTube to quickly find “Python tutorials”... and 30 minutes later you’re watching a cat playing piano or...

See all from Oviyan S

Recommended from Medium



 In Skill Stuff by CodeByUmar · Jun 8

7 Coding Patterns I Stole From Senior Engineers

Most developers do not become better because they learn more syntax. They become better because they stop making code harder than it needs...

★ 🖱️ 3.3K 💬 54 ↻ 60



ZERO BILLING

Run AWS
on your
laptop— free

No account. No credit card. No EC2 bill.

with Floci

\$0
local

AWS

EKS

S3

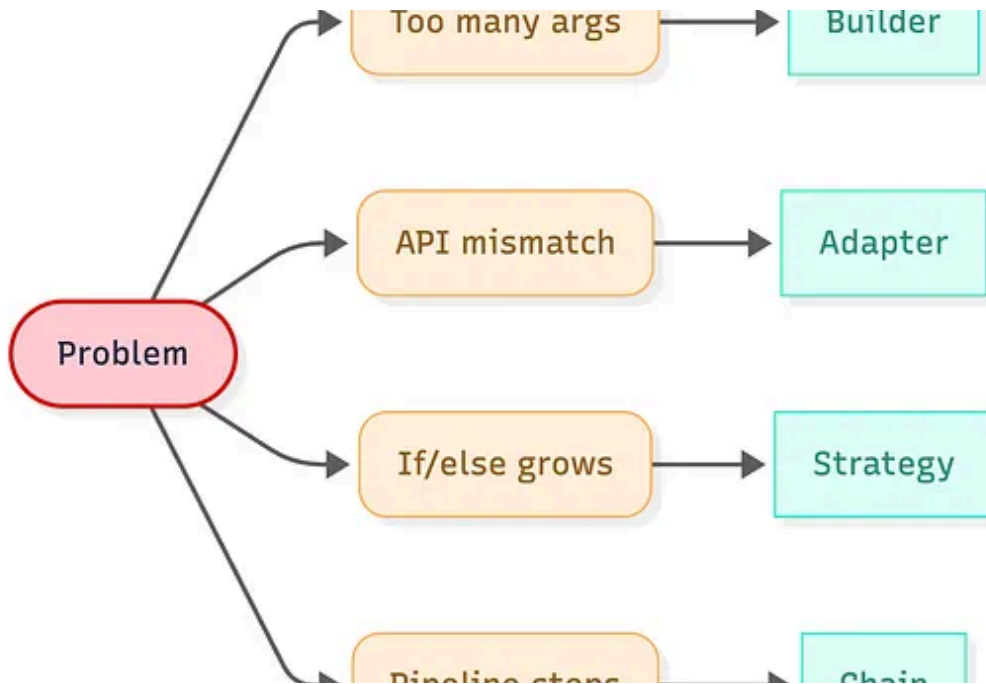
Azure

GCP

 In Cloud Wizards by Manoj Saini · Jun 18

Run AWS on your laptop — free

Introduction:



In Women in Technology by Alina Kovtun ★ · Jan 29

Stop Memorizing Design Patterns: Use This Decision Tree Instead

Choose design patterns based on pain points: apply the right pattern with minimal over-engineering in any OO language.

★ 🖱️ 8.8K 💬 90 ↺ 62



Anas Issath · Jan 11

Django Authentication & Authorization

The Security Foundation Every Django App Needs—Done Right from Day One

★ 🖱️ 250 💬 4 ↺ 2



Django

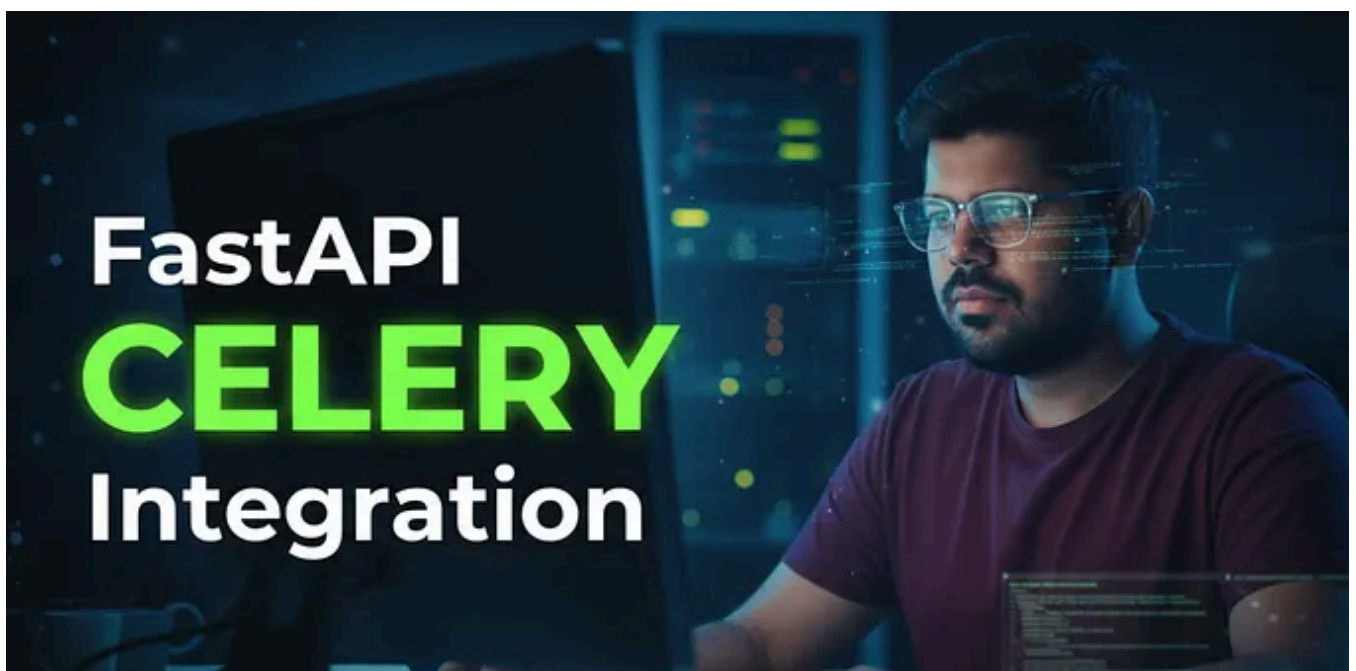


Priyanshu Rajput · Mar 27

🔥 Django REST Framework Serializers: The Backbone of Clean, Powerful APIs

If you've ever built APIs in Django, you've probably worked with serializers. And if you haven't fully understood them yet you're not...

★ 🖱️ 3





In AlgoMart by Yash Jain · Feb 1

Supercharging FastAPI with Celery: A Step-by-Step, Production-Ready Guide

Modern APIs are expected to be fast, responsive, and reliable—even when they trigger heavy work like sending emails, processing files, or...



79



See more recommendations